

Podstawy Uczenia Maszynowego

Czym się zajmiemy na ćwiczeniach?

~~Ćwiczenia 1: metody liniowe~~

~~Ćwiczenia 2: metody oparte na sąsiedztwie~~

~~Ćwiczenia 3: metody jądrowe~~

~~Ćwiczenia 4: metody probabilistyczne~~

Ćwiczenia 5: klasyfikacja niezbalansowana

Ćwiczenia 6: selekcja cech i redukcja wymiarowości

Ćwiczenia 7: klasteryzacja

Klasyfikacja niezbalansowana i anomaly detection

Klasyfikacja niezbalansowana

Klasyfikacja niezbalansowana

- technicznie rzecz biorąc, właściwie każdy zbiór jest niezbalansowany
- **w praktyce** mówimy o **klasyfikacji niezbalansowanej**, kiedy mamy znaczącą lub ekstremalną różnicę w liczności klas
- **imbalance ratio** - stosunek klasy pozytywnej do negatywnej, np. 1:99 (1 próbka klasy pozytywnej, 99 klasy negatywnej)
- wymaga **odpowiednich podejść** - to trudny problem!

Zastosowania klasyfikacji niezbalansowanej

- **usługi:**
 - churn prediction
- **finanse, cyberbezpieczeństwo:**
 - fraud detection
 - intrusion detection
- **przemysł (predictive maintenance):**
 - fault (malfunction) prediction
- **medycyna i farmacja:**
 - diagnoza chorób
 - klasyfikacja bioaktywnych molekuł (virtual screening)

Podejścia do klasyfikacji niezbalansowanej

- **cost-sensitive learning:**
 - dodajemy "wartość" danej klasy (koszt błędu) do algorytmu treningu
- **threshold tuning:**
 - obniżamy próg prawdopodobieństwa klasy pozytywnej z 50%
- **undersampling, oversampling:**
 - usunięcie przykładów z klasy większościowej (dominującej)
 - wygenerowanie sztucznych przykładów z klasy mniejszościowej
 - można je łączyć

Podejścia do klasyfikacji niezbalansowanej

- **anomaly detection:**
 - staramy się wykryć tylko nietypowe przykłady - **anomalie (anomalies)**
 - nazywane też **outlier detection** lub **novelty detection**
 - specjalne, dedykowane algorytmy
 - potrafi być bardzo szybkie i wydajne
 - do klasyfikacji ekstremalnie niezbalansowanej (ok. 1% klasy pozytywnej)

Cost-sensitive learning

Cost-sensitive learning

- **modyfikacja** istniejącego algorytmu, aby włączyć do niego **wagi klas (class weights)** i zwrócić większą uwagę na klasy mniejszościowe
- **zalety:**
 - najprostsze i najszybsze rozwiązanie
 - działa dla praktycznie wszystkich algorytmów (też dla sieci neuronowych)
- **wady:**
 - często samo to nie wystarcza przy bardzo mocnym niezbalansowaniu
 - zaburza przewidywane prawdopodobieństwa (gorsza kalibracja)

Wagi klas (class weights)

- można je dobrać ręcznie, walidacją skrośną albo heurystyką
- Scikit-learn proponuje heurystykę **"balanced"** z artykułu:

King, Gary, and Langche Zeng. *"Logistic regression in rare events data."* Political analysis 9.2 (2001): 137-163.

$$\text{class_weight}(y) = \frac{n}{K * |\text{class}(y)|}$$

n - liczba próbek

K - liczba klas

|class(y)| - liczność klasy w zbiorze treningowym

C.s. learning - algorytmy z funkcją kosztu

- jeżeli mamy **funkcję kosztu (loss function)**, to po prostu mnożymy wartość dla punktu przez wagę jego klasy:

$$L(\hat{y}, y) = \sum_{i=1}^n \text{class_weight}(y_i) * \text{loss}(\hat{y}_i, y_i)$$

- **algorytmy:**
 - regresja liniowa
 - regresja logistyczna
 - liniowy SVM
 - gradient boosting, np. XGBoost, LightGBM
 - drzewa decyzyjne
 - lasy losowe
 - sieci neuronowe

C.s. learning - kernel SVM

- C steruje "luzem", który dajemy granicy decyzyjnej przy danej próbce
- **bez ważenia** granica decyzyjna jest przesunięta dalej od klasy dominującej, bliżej klasy mniejszościowej
- **ważymy C** wagą klasy dla poszczególnych próbek
- "przepychamy" granicę z powrotem na środek, bliżej klasy większościowej

C.s. learning - kNN

- teoretycznie nie da się zrobić cost-sensitive learning w kNN, bo nie ma "learning"
- **ale:** da się zrobić ważoną klasyfikację
- ważne głosy (klasyfikacja) lub średnia ważona (regresja)

Naive Bayes a klasyfikacja niezbalansowana

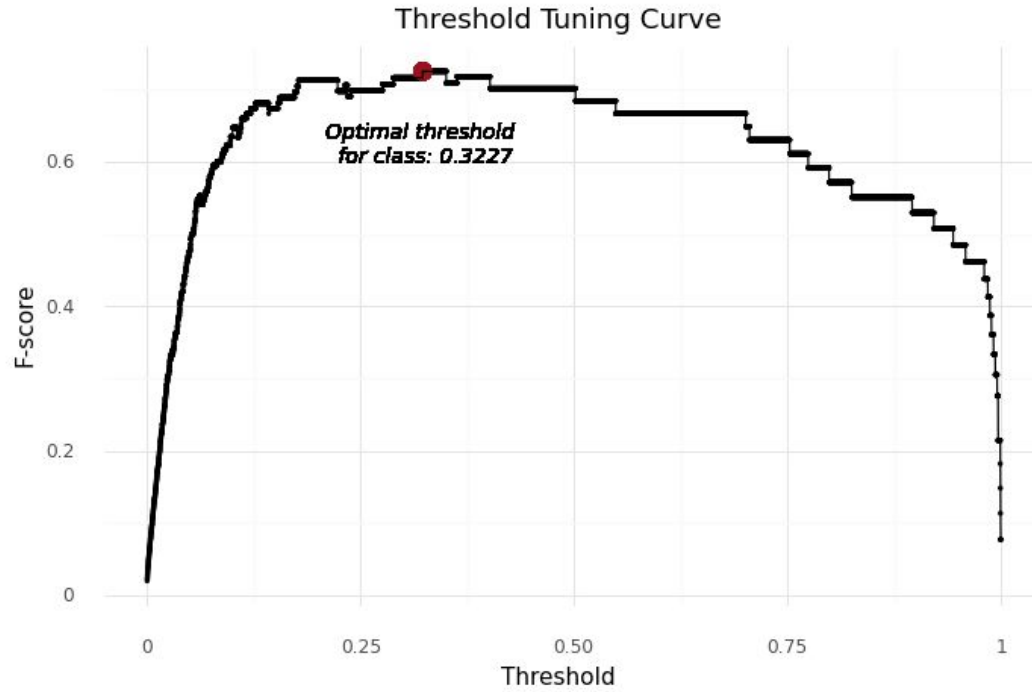
- klasyfikator naiwnego Bayesa **automatycznie działa dla klas niezbalansowanych**
- bo każdą klasę rozważa się całkowicie osobno
- nawet, jeżeli klasa mniejszościowa ma bardzo mało próbek, to dla Naive Bayesa to jest ok

Threshold tuning

Threshold tuning

- **próg klasyfikacji (classification threshold)** - przewidywane prawdopodobieństwo klasy pozytywnej, od którego zaliczamy do niej próbkę
- domyślnie 50%, ale **można go zmodyfikować**:
 - precision-recall tradeoff
 - **niższy** = wyższy recall, więcej FP (typowo to robimy)
 - **wyższy** = wyższe precision, więcej FN
- robimy dla **wytrenowanego** klasyfikatora
- odpowiada przesunięciu granicy decyzyjnej bliżej klasy większościowej
- nie działa dla metryk probabilistycznych, np. AUROC

Threshold tuning



Threshold tuning - zalety i wady

Zalety:

- prostota
- dobre wyniki
- łatwo zwizualizować tradeoff na metrykach

Wady:

- trzeba dobrać próg
- wymaga klasyfikatora, który potrafi przewidywać prawdopodobieństwa

Undersampling

Undersampling

- **obserwacja:** klasa większościowa nas generalnie nie interesuje i mamy jej dużo
- **undersampling** zwykle polega na jednym z **dwóch podejść**:
 - usunięcie mało ciekawych próbek klasy dominującej, z dala od potencjalnej granicy decyzyjnej
 - usunięcie zaszumionych próbek klasy dominującej blisko klasy mniejszościowej, które mogłyby "zepchnąć" granicę zbyt blisko niej
- oparte o metody najbliższych sąsiadów (wymaga normalizacji cech)
- **tylko na zbiorze treningowym!**

Random undersampling

- usuwamy **losowo** próbki klasy dominującej
- **zalety:** prostota, szybkość
- **wady:** nie bierze pod uwagę danych, może usunąć cenne punkty

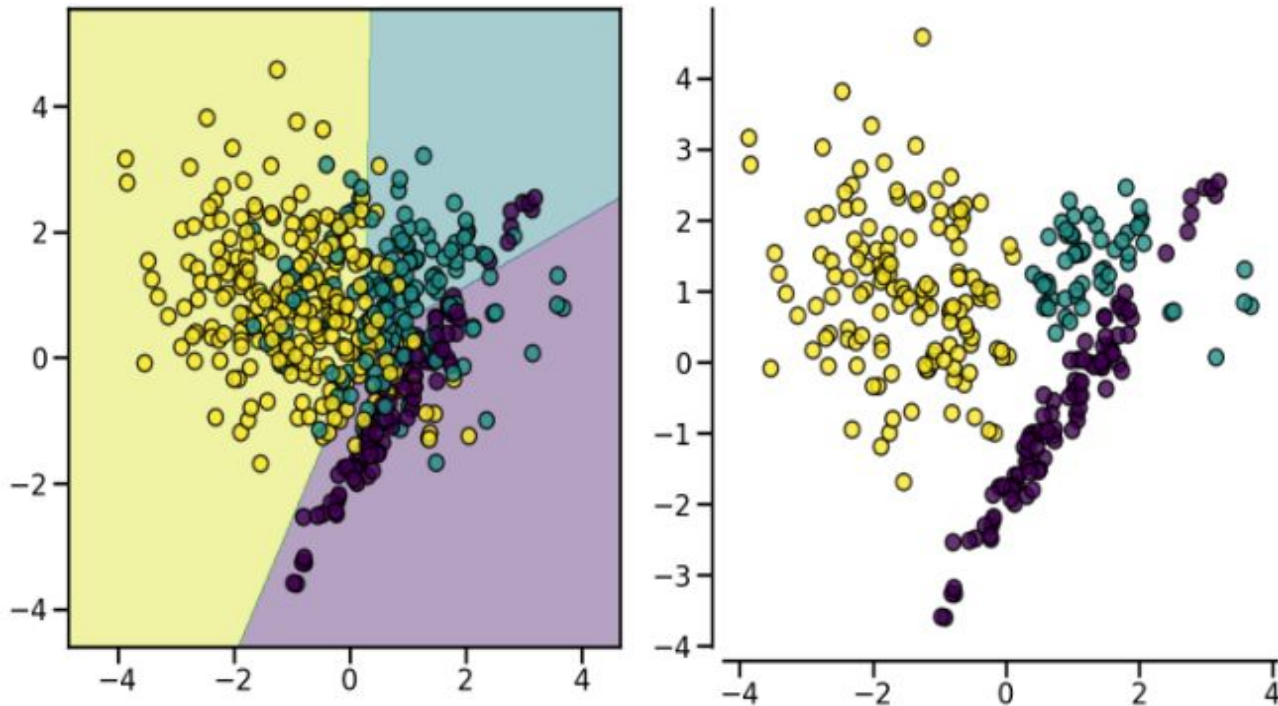
Edited Nearest Neighbor (ENN)

- **idea:** usunąć przykłady z klasy dominującej blisko granicy decyzyjnej
- zostawia tylko "pewne" przykłady, gdzie wszyscy sąsiedzi są z klasy dominującej
- **liczba sąsiadów k** steruje tym, jak ostro czyścimy - im większe k , tym agresywniej
- bardziej konserwatywnie - zostawiamy punkty, gdzie **moda** (najczęstsza klasa) sąsiadów to klasa dominująca

Algorytm:

1. Znajdź najbliższych sąsiadów dla klasy dominującej (domyślnie $k=3$)
2. Zostaw tylko te punkty, dla których wszyscy sąsiedzi są z tej samej klasy

Edited Nearest Neighbor (ENN) - wizualizacja



Edited Nearest Neighbor (ENN) - zalety i wady

Zalety:

- prostota
- "czyści" granicę decyzyjną
- szczególnie może pomóc klasyfikatorom skupionym na granicy decyzyjnej, np. kNN, SVM
- mocno lokalne

Wady:

- potrafi usuwać bardzo agresywnie
- koszt obliczeniowy
- klątwa wymiaru

Undersampling - zalety i wady

Zalety:

- "czyści" zbiór, usuwając przykłady redundantne, szum itp.
- zmniejsza koszt obliczeniowy dalszych algorytmów

Wady:

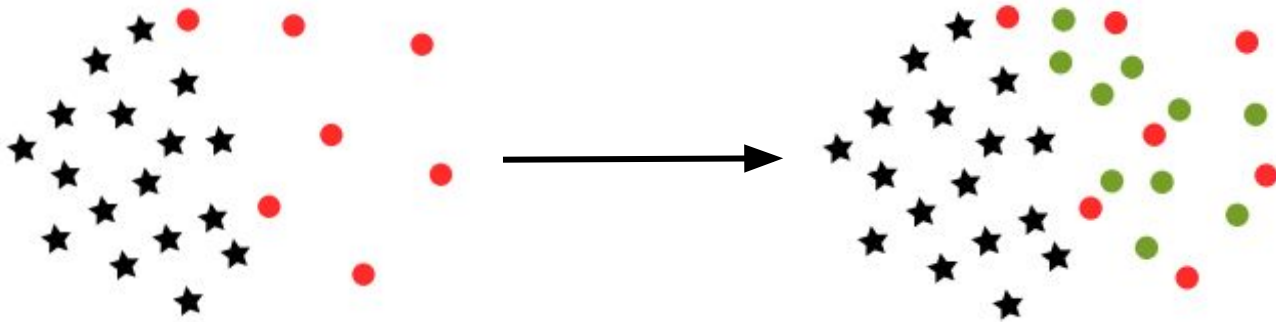
- zmniejsza ilość danych uczących - używamy tylko, gdy mamy ich dużo!
- koszt obliczeniowy
- może usunąć ważne próbki

Oversampling

Oversampling

- **obserwacja:** klasa mniejszościowa nas interesuje, chcielibyśmy jej mieć więcej
- **oversampling** polega na **generacji sztucznych przykładów**
- typowo przez **interpolację** próbek
- oparte o metody najbliższych sąsiadów (wymaga normalizacji cech)
- **tylko na zbiorze treningowym!**

Oversampling - wizualizacja



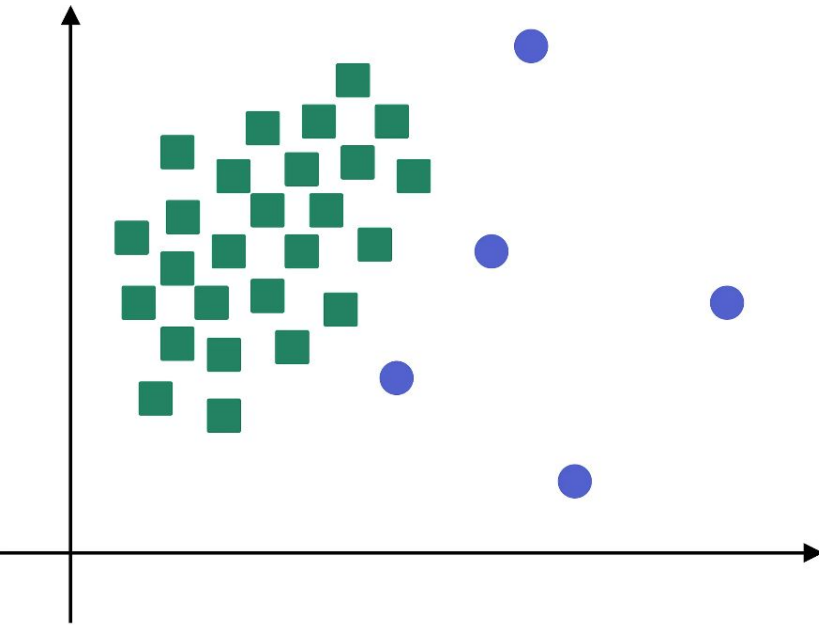
SMOTE

Synthetic Minority Oversampling TEchnique

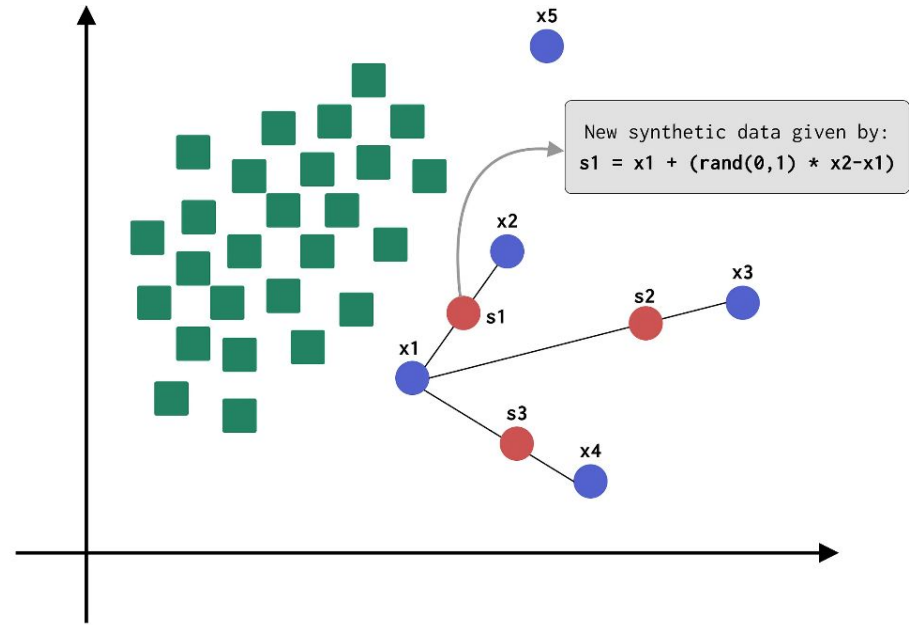
Algorytm:

1. Dla każdego punktu z klasy mniejszościowej znajdź jego k najbliższych sąsiadów z tej klasy (domyślnie $k=5$)
2. Wylosuj jednego z sąsiadów
3. Wylosuj punkt na odcinku między punktem a sąsiadem - to jest nowy wygenerowany punkt
4. Powtórz punkty 2-3, dopóki nie uzyskasz pożądanej liczby punktów

SMOTE - wizualizacja



a) Class Imbalance

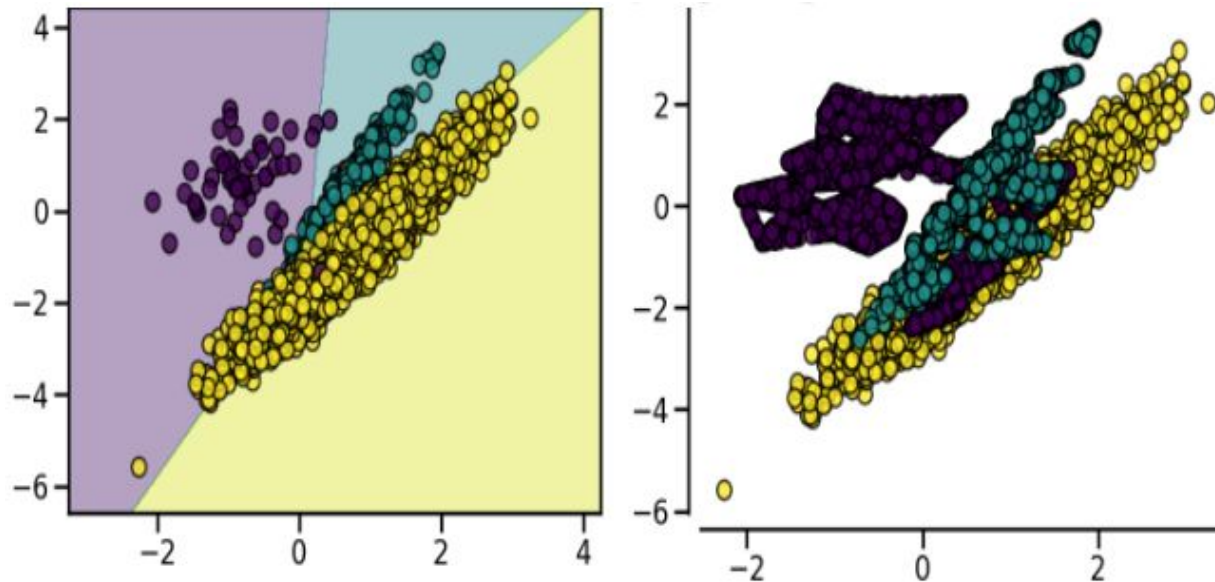


b) SMOTE

SMOTE

- typowo generuje się **wiele punktów** z każdego punktu (nawet wiele per odcinek), bo potrzebujemy dużo więcej klasy mniejszościowej
- dokonuje **interpolacji** klasy mniejszościowej - nie zmienia jej kształtu, tylko ją zagęści "w środku" chmury punktów
- **algorytm bazowy** dla zdecydowanej większości algorytmów oversamplingu
- **rozważa jednakowo** wszystkie punkty - rozwinięcia modyfikują tę część
- **hiperparametr k** - wpływa na różnorodność generowanych przykładów

SMOTE może oversamplować szum!



SMOTE - zalety i wady

Zalety:

- prostota
- szybkość - rozważamy tylko klasę mniejszościową
- łatwy do rozwijania, istnieje mnóstwo ulepszonych wersji

Wady:

- próbkuje punkty jednakowo
- może oversamplingować szum
- klątwa wymiaru
- tylko dane numeryczne

SMOTE - warianty

Kovács, György. *"An empirical comparison and evaluation of minority oversampling techniques on a large number of imbalanced datasets."* Applied Soft Computing 83 (2019): 105662.

"In this paper, a detailed, empirical comparison of **85 variants of minority oversampling techniques** is presented and discussed involving 104 imbalanced datasets for evaluation"

The implemented techniques: [SMOTE], [SMOTE_TomekLinks], [SMOTE_ENN], [Borderline_SMOTE1], [Borderline_SMOTE2], [ADASYN], [AHC], [LLE_SMOTE], [distance_SMOTE], [SMMO], [polynom_fit_SMOTE], [Stefanowski], [ADOMS], [Safe_Level_SMOTE], [MSMOTE], [DE_oversampling], [SMOBD], [SUNDO], [MSYN], [SVM_balance], [TRIM_SMOTE], [SMOTE_RSB], [ProWSyn], [SL_graph_SMOTE], [NRSBoundary_SMOTE], [LVQ_SMOTE], [SOI_CJ], [ROSE], [SMOTE_OUT], [SMOTE_Cosine], [Selected_SMOTE], [LN_SMOTE], [MWMOTE], [PDFOS], [IPADE_ID], [RWO_sampling], [NEATER], [DEAGO], [Gazzah], [MCT], [ADG], [SMOTE_IPF], [KernelADASYN], [MOT2LD], [V_SYNT], [OUPS], [SMOTE_D], [SMOTE_PSO], [CURE_SMOTE], [SOMO], [ISOMAP_Hybrid], [CE_SMOTE], [Edge_Det_SMOTE], [CBSO], [E_SMOTE], [DBSMOTE], [ASMOBD], [Assembled_SMOTE], [SDSMOTE], [DSMOTE], [G_SMOTE], [NT_SMOTE], [Lee], [SPY], [SMOTE_PSOBAT], [MDO], [Random_SMOTE], [ISMOTE], [VIS_RST], [GASMOTE], [A_SUWO], [SMOTE_FRST_2T], [AND_SMOTE], [NRAS], [AMSCO], [SSO], [NDO_sampling], [DSRBF], [Gaussian_SMOTE], [kmeans_SMOTE], [Supervised_SMOTE], [SN_SMOTE], [CCR], [ANS], [cluster_SMOTE], [SYMPROD], [SMOTEWB]

Oversampling - zalety i wady

Zalety:

- zwiększa ilość danych treningowych
- zwiększa wagę klasy mniejszościowej
- istnieją algorytmy zagęszczające tylko kluczowe obszary

Wady:

- może powodować overfitting, szczególnie, gdy wygenerujemy za dużo
- interpoluje - nie zmieni fundamentalnie przestrzeni
- klątwa wymiaru

Podejście hybrydowe

Podjęcie hybrydowe

- podejścia te można połączyć w **algorytm hybrydowy**
- **elementy:** oversampling, undersampling, cost-sensitive learning, threshold tuning
- np. SMOTE + ENN, cost-sensitive learning + threshold tuning

Zalety:

- elastyczność
- łączy zalety i zwalcza wady under- i oversamplingu

Wady:

- złożone rozwiązanie
- koszt obliczeniowy
- dodatkowe hiperparametry

Metryki w klasyfikacji niezbalansowanej

Metryki binarne

- **precyzja (precision):**

$$Precision = \frac{TP}{TP + FP}$$

- **czułość (recall):**

$$Recall = \frac{TP}{P} = \frac{TP}{TP + FN}$$

- **F1-score:**

$$F1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

	Actual 1	Actual 0
Predicted 1	True Positive (TP)	False Positive (FP)
Predicted 0	False Negative (FN)	True Negative (TN)

	Actual 1	Actual 0
Predicted 1	True Positive (TP)	False Positive (FP)
Predicted 0	False Negative (FN)	True Negative (TN)

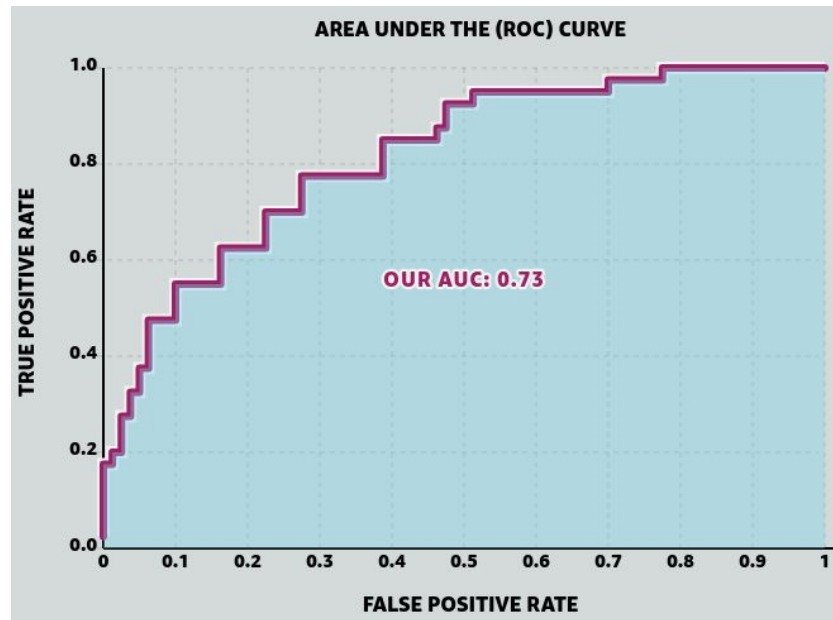
	Actual 1	Actual 0
Predicted 1	True Positive (TP)	False Positive (FP)
Predicted 0	False Negative (FN)	True Negative (TN)

Metryki binarne

- "dobre" wartości **zależą** od poziomu niezbalansowania
- **powód:** kiedy klasy pozytywnej jest bardzo mało, każdy FP i FN (szczególnie) mają bardzo mocny efekt
- F1-score rzędu 50% może być już doskonały np. w finansach

AUROC

- [tutorial o ROC i AUROC](#)
- Area Under Receiver Operating Characteristic
- "The AUROC is the probability that the model will rank a randomly chosen positive example more highly than a randomly chosen negative example."
- bierze pod uwagę **tylko prawdopodobieństwo klasy pozytywnej**
- dobre przy umiarkowanym niezbalansowaniu



Problem z AUROC

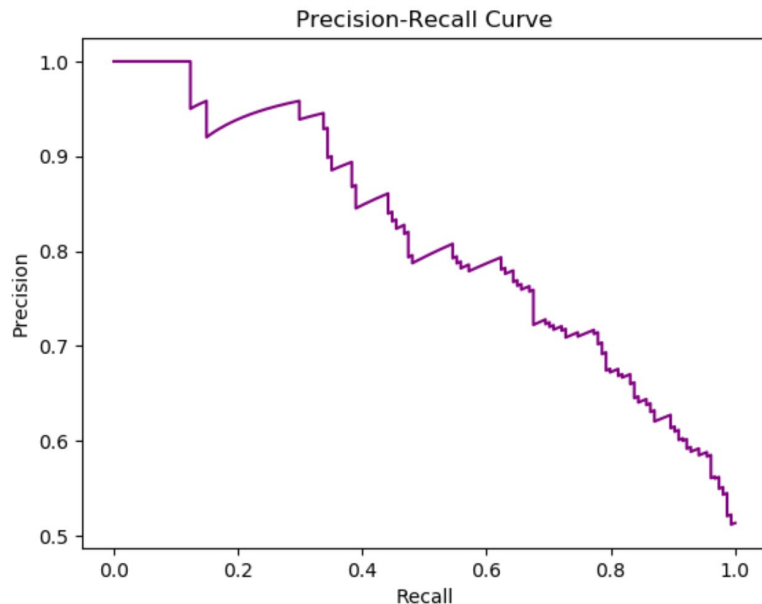
- AUROC **nie działa dobrze**, kiedy mamy **ekstremalne niezbalansowanie** klas
- **problem:** AUROC używa false positive rate (FPR)

$$FPR = \frac{FP}{FP + TN}$$

- nie działa dobrze, gdy mamy dominację TN
- **powód:** "dużo" FP to np. 100 (bo klasy pozytywnej jest mało), a TN mamy rzędu 10000+
- **chcemy:** zignorować TN oraz użyć prawdopodobieństw

Precision-recall curve i AUPRC

- **Precision-Recall Curve (PRC)** to krzywa z precyzji i czułości dla malejących progów klasyfikacji
- **Area Under Precision-Recall Curve (AUPRC)** to pole pod tą krzywą
- czasem nazywane **Average Precision (AP)**
- ma dużo różnych równoważnych sformułowań
- podobne do ROC i AUROC, ale skupia się jeszcze mocniej na klasie pozytywnej



AUPRC - cechy

- **specyficzny zakres wartości**
- minimum to **ułamek klasy pozytywnej** w zbiorze, maximum to 1
- **uwaga** na under- i oversampling, bo zmieniają proporcję klasy pozytywnej!
- dobre wartości bardzo zależą od zbioru, ale zwykle są niskie
- np. 0.3 może być doskonałe
- **zalety:** nie używa TN, używa prawdopodobieństw
- **wada:** specyficzny zakres wartości, słabo się "sprzedaje" w artykułach, prezentacjach itp.

Własności AUPRC

"A Closer Look at AUROC and AUPRC under Class Imbalance" M. McDermott et al.

- *"AUPRC is explicitly discriminatory in favor of high-scoring subpopulations"*
- *"AUPRC prioritizes high-score mistakes, AUROC treats all mistakes equally"*
- **wnioski:**
 - AUPRC **nie** działa ogólnie lepiej niż AUROC w klasyfikacji niezbalansowanej
 - AUPRC skupia się na **najbardziej prawdopodobnych** segmentach danych
 - tj. najwyższe średnie prawd. klasy pozytywnej
- dobre AUPRC = mało FP, dobre predykcje najwyższych prawd.
- AUPRC jest dobre np. do wyszukiwania (information retrieval), rankowania
- **sugestia autorów:** AUROC ogólnie przydatne, AUPRC kiedy wybieramy top-k przykładów

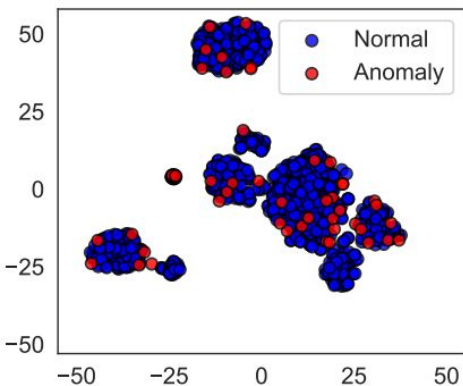
Anomaly detection

Anomaly detection

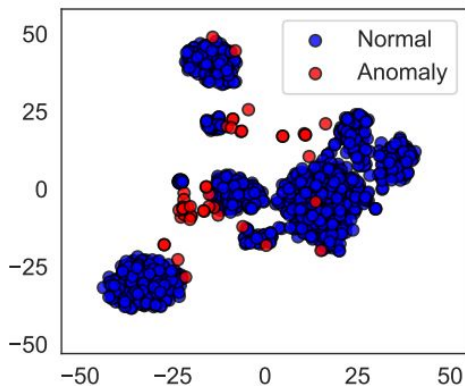
- **anomalie (anomalies)** to punkty tak różne od "typowych", że trzeba uznać, że zostały wygenerowane przez inne zjawisko
- można je traktować jak ekstremalnie małą klasę pozytywną
- wiodący benchmark w anomaly detection to ADBench, od autorów biblioteki PyOD

Han, Songqiao, et al. *"Adbench: Anomaly detection benchmark."* Advances in Neural Information Processing Systems 35 (2022): 32142-32159.

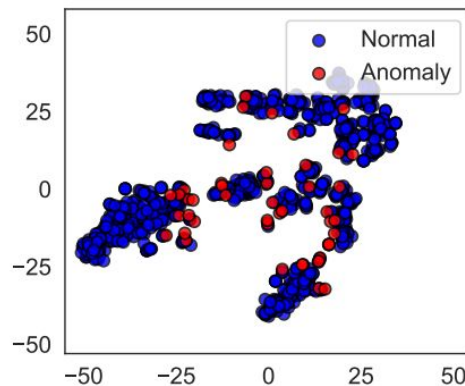
Rodzaje anomalii



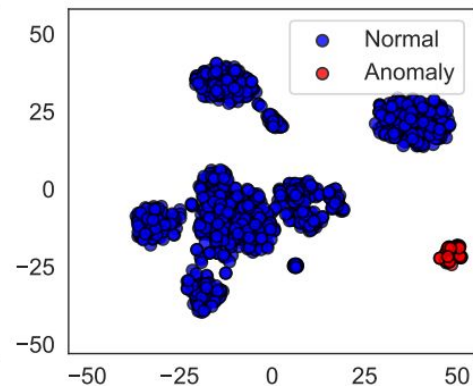
(a) Local



(b) Global



(c) Dependency



(d) Clustered

- **local** - anomalia różni się od pobliskich, jest nietypowa względem "podobnych" punktów
- **global** - anomalia różni się od ogółu danych, jest inna od "normalnych" punktów
- **dependency** - zależności między typowymi wartościami cech są zaburzone
- **clustered** - anomalie są do siebie podobne i tworzą zbite klastry

Anomaly detection i klasy

- typowe podejścia do anomaly detection są **nienadzorowane (unsupervised)** i mierzą "anormalność" punktu, bez patrzenia na klasy
- do oceny jakości dalej trzeba oznaczonych punktów, przynajmniej z klasy pozytywnej
- podejścia **nadzorowane (supervised)** to klasyfikacja z modyfikacjami - typowo lepsze wyniki, ale dużo większy koszt stworzenia zbioru
- algorytmy **pół-nadzorowane (semi-supervised)** łączą powyższe na różnorodne sposoby

Outlier score

- **outlier score** wyznacza "nietypowość" punktu według algorytmu
- **zakres wartości** zależy od algorytmu
- biblioteki często **normalizują** takie wyniki, żeby dało się to porównywać, np. PyOD

Contamination

- typowy **hiperparametr** metod outlier detection, oczekiwana ilość anomalii w zbiorze
- **contamination**, w zakresie (0, 0.5)
- używany, żeby ustalić **próg (threshold)** outlier score, powyżej którego klasyfikujemy anomalię:
 - obliczamy outlier scores w zbiorze treningowym
 - sortujemy malejąco
 - wybieramy wartość dla percentyla równego contamination (np. 10%)
 - to jest próg klasyfikacji

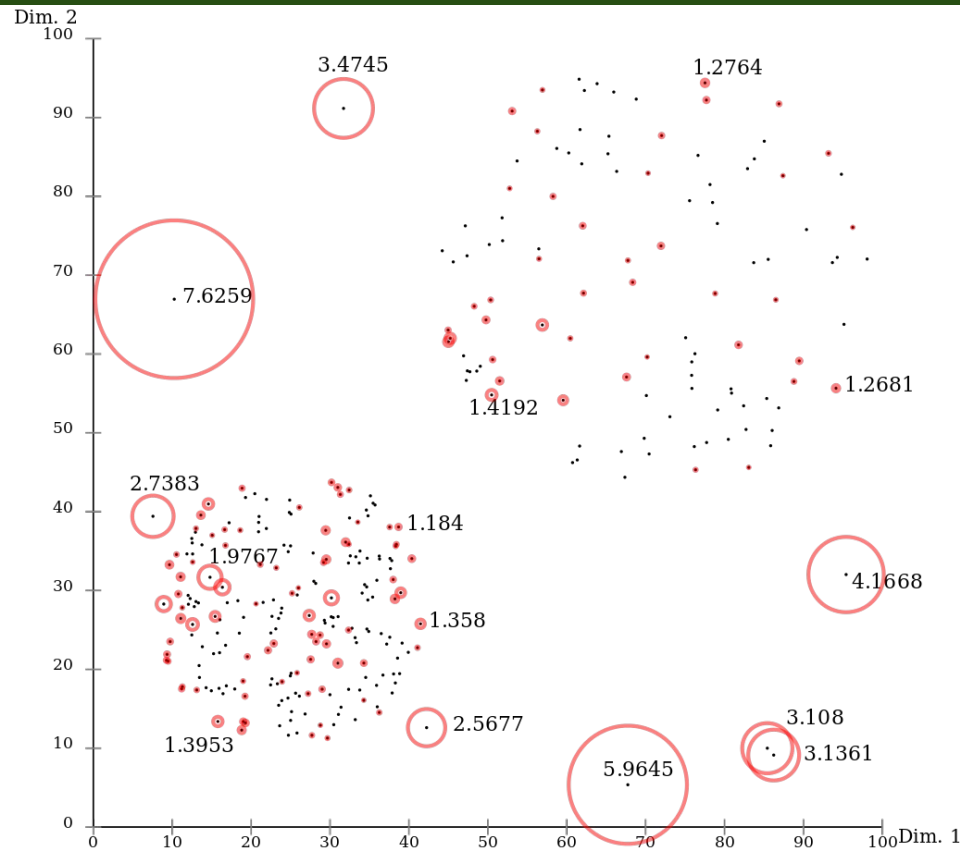
Algorithmy anomaly detection

K najbliższych sąsiadów (kNN)

- **idea:** jeżeli najbliżsi sąsiedzi są daleko, to mamy outlier
- metoda **oparta o bliskość (proximity-based)**
- **outlier score:** odległość do k-tego najbliższego sąsiada
- **problem:** nie bierze pod uwagę gęstości sąsiadów, jakiś obszar może być po prostu rzadki, ale normalny

kNN - problem z rzadkimi regionami

kNN uznałby prawy górny klaster za anomalie!



K najbliższych sąsiadów (kNN) - zalety i wady

Zalety:

- prostota
- przeciętnie świetne wyniki

Wady:

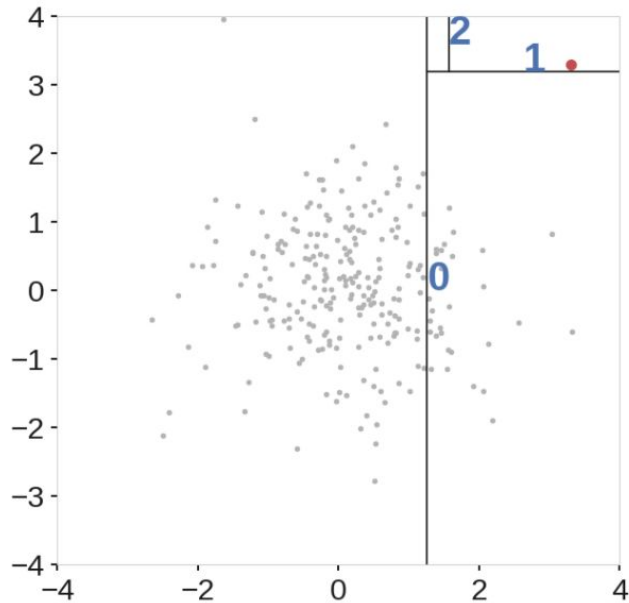
- koszt obliczeniowy (ale: ANN)
- klątwa wymiaru

Rodzaj anomalii	Jakość
Lokalne	Świetne
Globalne	Świetne
Dependency	Świetne
Klastrowane	Słabe

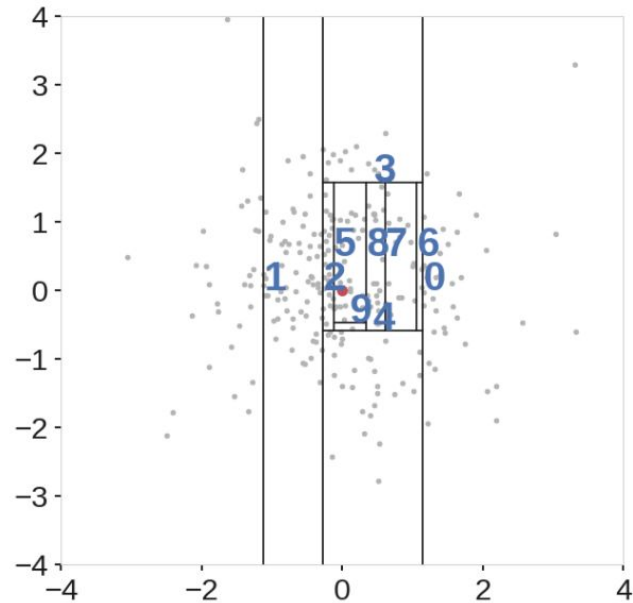
Isolation Forest

- **idea:** anomalie powinny mieć "dziwne" wartości cech, odróżniające je od zwykłych punktów
- budujemy **losowe drzewa decyzyjne** - wybieramy losowo cechę i punkt podziału przy konstrukcji drzewa
- budujemy las drzew, każde na **losowym podzbiorze** próbek
- można też wybierać podzbiór cech (jak w Random Forest), jak mamy za dużo cech
- "anormalność" punktu wyznacza **średnia długość ścieżki**, której potrzebujemy, żeby dojść do liścia w drzewie
- krótkie ścieżki = punkt jest **łatwy do odizolowania**

Isolation Forest - wizualizacja

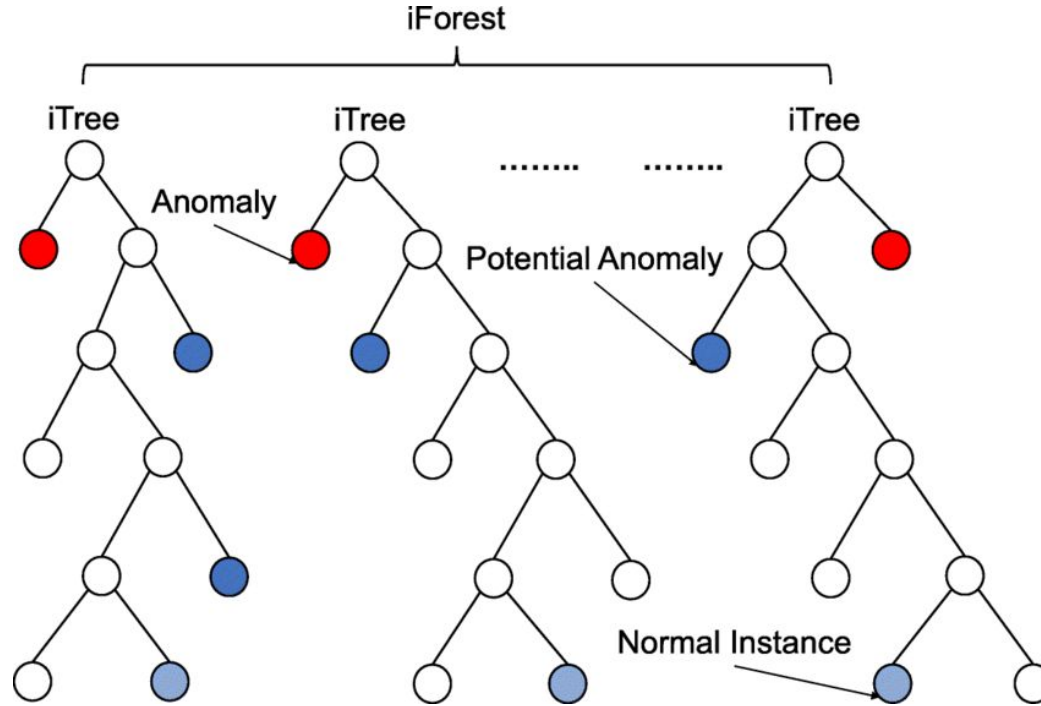


(a) Anomaly point



(b) Nominal point

Isolation Forest - wizualizacja



Isolation Forest

- **problem:** długość ścieżki jest nieznormalizowana i bardzo losowa (podzbiór, punkty podziału itp.)
- **idea:** znormalizować przez teoretyczną średnią długość ścieżki do liścia w drzewie BST

$$c(\psi) = 2H(\psi - 1) - \frac{2(\psi - 1)}{n}$$

ψ - wielkość próbki dla drzewa (np. 100, 256)

H - liczba harmoniczna, $H(x) \approx \ln(x) + 0.57721$ (stała Eulera)

Isolation Forest

- **outlier score:**

$$\text{outlier_score}(x) = 2^{-\frac{\text{avg}(h(x))}{c(\psi)}}$$

$\text{avg}(h(x))$ - średnia długość ścieżki według wytrenowanych drzew

- **zakres wartości:** od 0 do 1
- im wyżej, tym bardziej "anormalny" przykład
- 0.5 odpowiada "przeciętnym" punktom

Isolation Forest - hiperparametry

- **liczba próbek w podzbiorach:**
 - typowo kilkaset
 - robi undersampling, mała liczba próbek nawet pomaga
- **liczba drzew:**
 - typowo kilkaset
 - zwykle asymptotycznie poprawia wynik aż do ok. 500-1000 drzew

Isolation Forest - złożoność obliczeniowa

- trening to konstrukcja losowych drzew na małych losowych podzbiorach
- predykcja to przejście po tych drzewach i zapamiętanie finalnej głębokości
- idealnie uwspółbieżnialne w obu przypadkach

- **złożoność treningu:**
 - dla bardzo dużych danych $O(n)$, w praktyce często subliniowa, <1 sekunda
- **złożoność predykcji:**
 - taka, jak treningu, ale z większą stałą, kilka-kilkanaście sekund

Isolation Forest - zalety i wady

Zalety:

- bardzo niski koszt obliczeniowy
- doskonała skalowalność
- dobrze działa dla danych wysokowymiarowych
- działa nawet wtedy, kiedy dane treningowe nie zawierają żadnych anomalii

Rodzaj anomalii	Wyniki
Lokalne	Słabe
Globalne	Świetne
Dependency	Słabe
Klastrowane	Świetne

Wady:

- podziały ortogonalne - dla niektórych zbiorów może zadziałać bardzo słabo

XGBoost Outlier Detection (XGBOD)

- **idea:** połączyć zwykłe cechy i "anormalność" według różnych algorytmów
- używamy **outlier scores jako dodatkowych cech**, np. z kNN czy Isolation Forest
- na tym robimy normalną klasyfikację, używając XGBoost
- daje to **XGBoost Outlier Detection (XGBOD)**
- **zalety:** doskonałe wyniki
- **wady:** dużo hiperparametrów, koszt obliczeniowy, wymaga pełnej znajomości klas

Slajdy bonusowe

Inne algorytmy undersamplingu

- [Condensed Nearest Neighbor](#) - najstarszy algorytm undersamplingu, usuwa punkt z klasy dominującej, jeżeli jego najbliższy sąsiad jest z klasy mniejszościowej
- [RepeatedENN](#) - powtarzanie ENN wielokrotnie, aż do braku zmian (jak w CNN)
- [NearMiss](#) - oparte bezpośrednio o kNN, pozwala kontrolować ilość usuwanych przykładów
- [Tomek's links](#) - znajduje pary punktów z przeciwnych klas, które są dla siebie wzajemnie najbliższym sąsiadem (tzw. Tomek's links) i usuwa punkt z klasy dominującej

Local Outlier Factor (LOF)

- **idea:** trzeba sprawdzić gęstość w odniesieniu do gęstości najbliższych sąsiadów
- metoda **oparta o gęstość (density-based)**
- $k\text{-distance}(A)$ - odległość k-tego najbliższego sąsiada punktu A
- $N_k(A)$ - sąsiedzi bliżsi lub równi k-temu (ponad k, gdy mamy identyczne odległości)
- odległość **od punktu B do punktu A** to tzw. **reachability distance**:
$$\text{reachability-distance}_k(A, B) = \max\{k\text{-distance}(B), d(A, B)\}$$
- mówi nam to, jak łatwo jest dojść do A od jakiegoś innego punktu
- max wygładza odległości

Local Outlier Factor (LOF)

- **local reachability density (lrd)** mówi, jak średnio łatwo dojść do A od jego najbliższych sąsiadów:

$$lrd_k(A) = 1 / \left(\frac{\sum_{B \in N_k(A)} reachability-distance(A, B)}{|N_k(A)|} \right)$$

- **Local Outlier Factor (LOF)** to stosunek tego, jak średnio łatwo dojść do sąsiadów względem tego, jak łatwo dojść od nich do punktu A:

$$LOF_k(A) = \frac{\sum_{B \in N_k(A)} \frac{lrd_k(B)}{lrd_k(A)}}{|N_k(A)|} = \frac{\sum_{B \in N_k(A)} lrd_k(B)}{|N_k(A)| * lrd_k(A)}$$

- miara lokalnej gęstości punktu A względem gęstości jego sąsiadów

Local Outlier Factor (LOF)

$$LOF_k(A) = \frac{\sum_{B \in N_k(A)} \frac{lrd_k(B)}{lrd_k(A)}}{|N_k(A)|} = \frac{\sum_{B \in N_k(A)} lrd_k(B)}{|N_k(A)| * lrd_k(A)}$$

- $LOF < 1$: A ma większą gęstość od sąsiadów (**inlier**)
- $LOF \sim 1$: A ma taką samą gęstość, co sąsiedzi
- $LOF > 1$: A ma mniejszą gęstość od sąsiadów - outlier?
- **ciężko dobrać próg**: Scikit-learn przyjmuje domyślnie 1.5 jako próg outliera, ale to zależy od zbioru

Local Outlier Factor (LOF) - zalety i wady

Zalety:

- lepszy od kNN do anomalii lokalnych

Wady:

- duża czułość na próg
- koszt obliczeniowy
- klątwa wymiaru

Rodzaj anomalii	Wyniki
Lokalne	Świetne
Globalne	Przeciętne
Dependency	Świetne
Klastrowane	Całkowicie nie działa

Warianty Isolation Forest

- **Extended Isolation Forest** ([artykuł](#)) - robi podziały losowymi płaszczyznami (trochę jak Random Projection Forest), zamiast równoległymi do osi
- **SCiForest** ([prezentacja](#), [artykuł](#)) - podziały hiperpłaszczyznami, które nie są losowe, tylko obliczane; w założeniach ma lepiej działać dla clustered anomalies
- zaimplementowane w bibliotece [isotree](#) - bardzo zoptymalizowanej, ale niekoniecznie prostej w instalacji i z "pythonic" API

Inne ważne algorytmy anomaly detection

- **Kernel Density Estimation (KDE):**
 - estymujemy rozkład klasy normalnej
 - niskie log-likelihood dla nowego przykładu oznacza anomalię
- **One-Class SVM (OCSVM):**
 - SVM, który buduje hiperpłaszczyznę tylko wokół klasy negatywnej
 - jak coś leży poza nią, to jest klasą pozytywną
 - bardzo kosztowny przez kernele - często robi się kernel approximation